

Reviewer Pack — Minimal Order of Permutation Groups and Communication Comple...

Entry f8bce94092a1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 06:27:19 UTC

Minimal Order of Permutation Groups and Communication Complexity Rank Variance

Entry ID: f8bce94092a1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 06:27:19 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Permutation Groups) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For a given Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal order of a permutation group that can be used to compute f in parallel with communication complexity rank $R(f)$ is $\Theta(R(f)^2)$.

Rationale (proposer's reasoning):

The structure of permutation groups can potentially capture complex interactions between bits in Boolean functions. By studying the minimal order of permutation groups, we may uncover new insights into the intrinsic complexity of computation and communication.

Taxonomy category: group_theory_to_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 852e27c9b31154b5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal order of a permutation group for computing Boolean function f is within a factor of 1.5 from the square of its communication complexity rank $R(f)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order permutation groups" AND "communication complexity rank variance" - "permutation group computation Boolean function" AND "matrix rank" - "parallel computation Boolean functions" AND "permutation group order communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity_rank(f):
        n = len(next(iter(f.keys())))
        rank = 0
        for t in f:
            row = [f[tuple([1 if j == i else 0 for j in range(n)])] for t in f]
            rank = max(rank, sum(row))
        return rank

    def permutation_group_order(f):
        n = len(next(iter(f.keys())))
        # Simplified example: use the identity permutation
        return 1

    instances_tested = 30
    n_max = 40
    metric_values = []

    for _ in range(instances_tested):
        n = random.randint(5, n_max)
        f = {tuple(random.choice([0, 1]) for _ in range(n)): random.choice([0, 1]) for _ in range(n)}

        rank = communication_complexity_rank(f)
        order = permutation_group_order(f)

        metric_values.append(order / (rank ** 2))

    mean_value = sum(metric_values) / instances_tested
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / instances_tested)
```

```

conjecture_holds = all(0.5 <= v <= 1.5 for v in metric_values)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Order/CommunicationComplexityRank^2",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition or falsification condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13275
2	preregistration	ollama_remote	glm4:latest	0	0	14635
3	novelty	ollama_remote	glm4:latest	0	0	8482
4	novelty	ollama_remote	glm4:latest	0	0	8938
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13880
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10776
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8797
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8233
9	judge	ollama_remote	glm4:latest	0	0	47999

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 135016 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/f8bce94092a1.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f8bce94092a1.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f8bce94092a1.tar.gz* (if generated)